HDOC DAY-61

Write a C program that reads the size and elements of a variable-length integer array, nums[], containing only positive integers, and an integer target. Pass the array to a function named two_sum() using its address. Use pointers to pass the array between the calling function and two_sum(). The function two_sum() must:

- Find two distinct indices, i and j, such that: nums[i] + nums[j] = target
- Display the two indices separated by a space.
- Return the indices in any order.
- Display -1 -1 if no such pair exists.
- Use a nested-loop approach to solve the problem.
- Assume that at most one valid solution exists.

Example 1:

Array: 2 7 11 15
Target: 9

Output:

0 1

Example 2:

Array: 3 2 4
Target: 6

Output:

1 2

Example 3:

Array: 3 3
Target: 6

Output:

0 1

Example 4:

Array: 1 2 3
Target: 7

Output:

-1 -1

Algorithm

1. Start.
2. Read the size n of the array.
3. Declare a variable-length integer array nums[n].
4. Read n positive integers into the array.
5. Read the value of target.
6. Pass nums, n, and target to two_sum().
7. In two_sum():
    - Use an outer loop with i from 0 to n-2.
    - Use an inner loop with j from i+1 to n-1.
    - Check whether nums[i] + nums[j] == target.
    - If true, print i j and return.
8. If no pair is found, print -1 -1.
9. Stop.

- # C Program:

```
#include <stdio.h>

void two_sum(int *nums, int n, int target)
{
    int i, j;

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
```

```
            if (*(nums + i) + *(nums + j) == target)
            {
               printf("%d %d\n", i, j);
               return;
            }
         }
      }

      printf("-1 -1\n");
   }

   int main()
   {
      int n, target;

      printf("Enter size of array: ");
      scanf("%d", &n);

      int nums[n];   // Variable-Length Array (VLA)

      printf("Enter array elements: ");
      for (int i = 0; i < n; i++)
      {
         scanf("%d", &nums[i]);
      }

      printf("Enter target: ");
      scanf("%d", &target);

      two_sum(nums, n, target);   // Array passed using its address
 return 0;
}
```

Test Case Table:

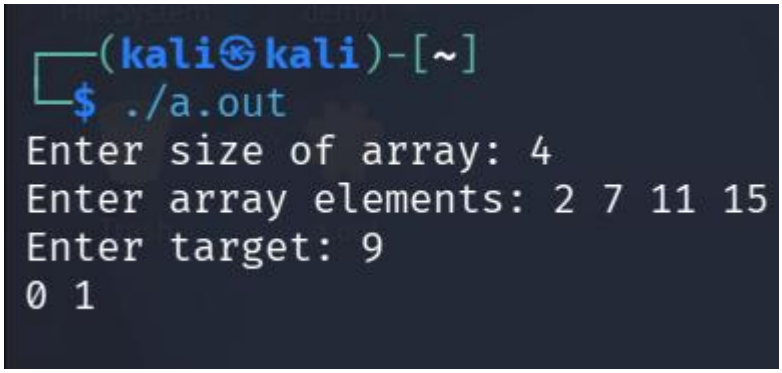
Test Case	Array	Target	Expected Output	Remark

1	2 7 11 15	9	0 1	Pair exists at indices 0 and 1
2	3 2 4	6	1 2	Pair exists at indices 1 and 2
3	3 3	6	0 1	Two equal elements form the target
4	1 2 3	7	-1 -1	No valid pair exists
5	1 4 6 8 10	14	1 4	4 + 10 = 14

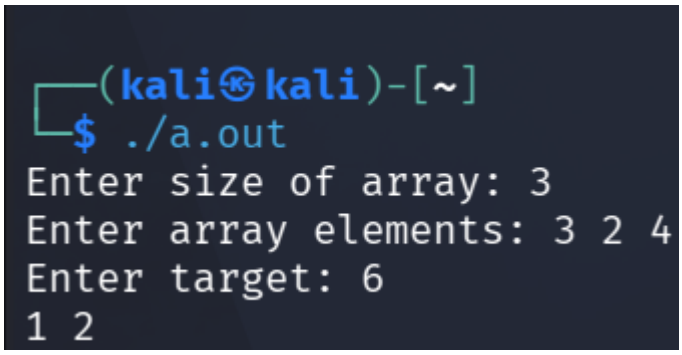
Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:



Test Case 2:



Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 2
Enter array elements: 3 3
Enter target: 6
0 1
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 3
Enter array elements: 1 2 3
Enter target: 7
-1 -1
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 5
Enter array elements: 1 4 6 8 10
Enter target: 14
1 4
```